

ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution

26. 02. 25

황규상



Background:

❖ Large Language Models (LLMs)

- ✓ LLM은 대규모 언어 모델(Large Language Model)의 약자로,
텍스트 데이터를 학습하여 인간처럼 언어를 이해, 생성, 요약, 번역하는 인공지능(AI) 모델
문맥을 파악하고 다음 단어를 예측하는 방식으로 작동하며, 챗봇, 콘텐츠 생성 등 다양한 자연어 처리 작업에 쓰임
(ex: GPT, Gemini)

❖ Hyper-Heuristics (HHs)

- ✓ 하이퍼 휴리스틱은 개별 문제에 대한 해(Solution)를 직접 찾는 대신, '해를 찾는 방법(Heuristic)' 자체를 탐색하거나 생성하는 상위 수준의 방법론

❖ LanguageHyper-Heuristics (LHH)

- ✓ 이 논문에서 제안한 개념으로 거대 언어 모델(LLM)의 등장으로 나타난 하이퍼 휴리스틱의 새로운 변종, 휴리스틱 생성 과정에 LLM을 활용함

❖ ReEvo: Reflective Evolution

- ✓ ReEvo는 언어 하이퍼 휴리스틱(LHH)의 성능을 극대화하기 위해 제안된 프레임워크
- ✓ 진화 연산의 효율적인 탐색 능력과 LLM의 성찰(Reflection) 능력을 결합한 방법론

❖ Pure LHH

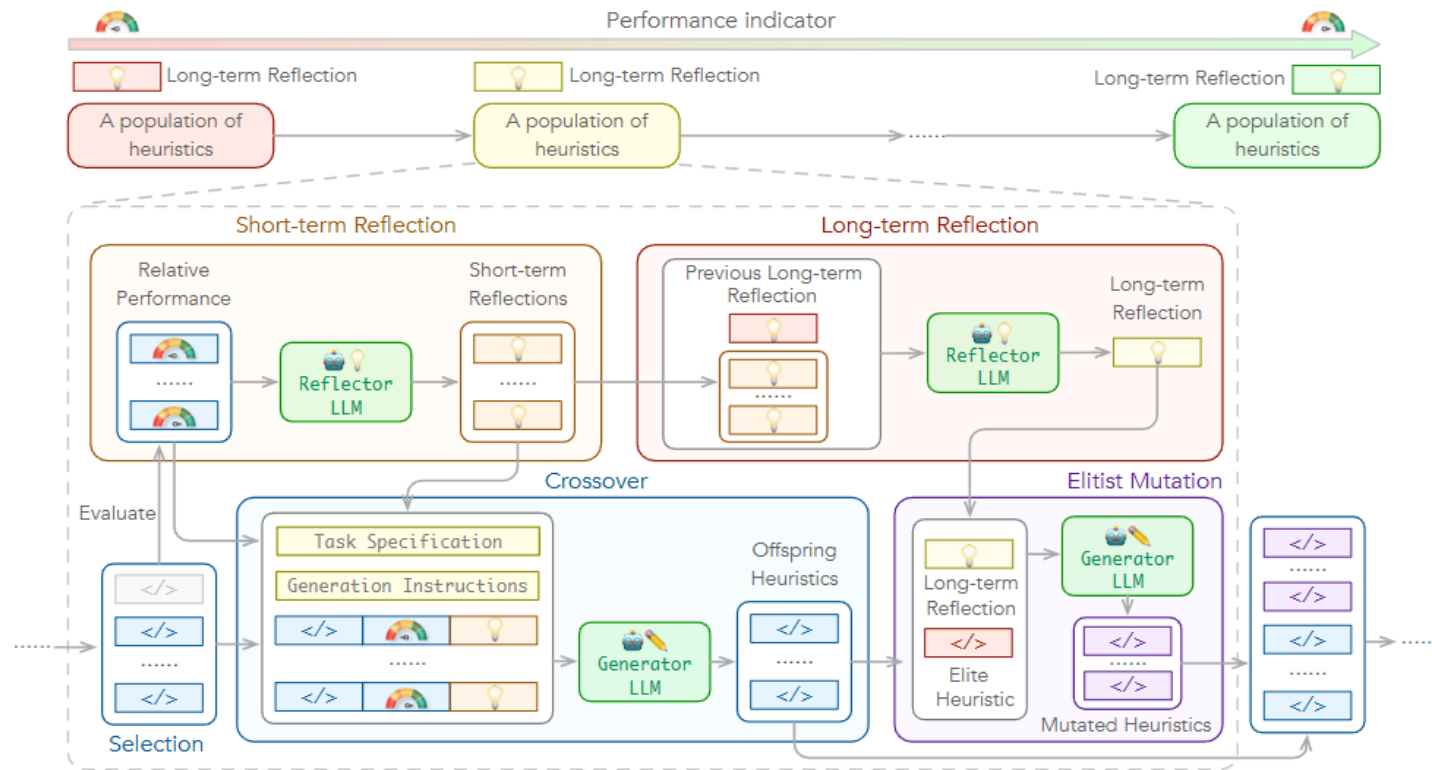
- ✓ 단순 LLM 코드 생성 방식은 비효율적인 샘플과 블랙박스 COP에 대해 제한된 추론 능력

❖ ReEvo

- ✓ 단순 LHH보다 훨씬 적은 횟수의 코드 평가(100회 미만)만으로도 최적의 해를 찾아냄
- ✓ 5가지 알고리즘 유형과 6가지의 서로 다른 조합 최적화 문제에서 SOTA성능을 기록
- ✓ 문제에 대한 정보가 완벽할 때뿐만 아니라, 문제 내용을 가린 블랙박스 상황에서도 LLM이 스스로 로직을 추론, 성능을 개선

III ReEvo(Reflective Evolution)

❖ ReEvo(Reflective Evolution)



(a) ReEvo pipeline. **Top:** ReEvo evolves a population of heuristics. Insights and knowledge are verbalized as long-term reflections and accumulated throughout iterations. **Bottom:** A ReEvo iteration contains five sequential steps: selection, short-term reflection, crossover, long-term reflection, and elitist mutation.

❖ Population initialization:

- ✓ 먼저 문제의 명세(COP 설명, 휴리스틱 종류, 휴리스틱 기능 포함) 기반으로 LLM이 여러 개의 초기 휴리스틱 코드를 생성. 추가적으로 전문가가 만든 코드나 간단한 규칙(Seed Heuristics)을 예시로 제공.

❖ Generator LLM:

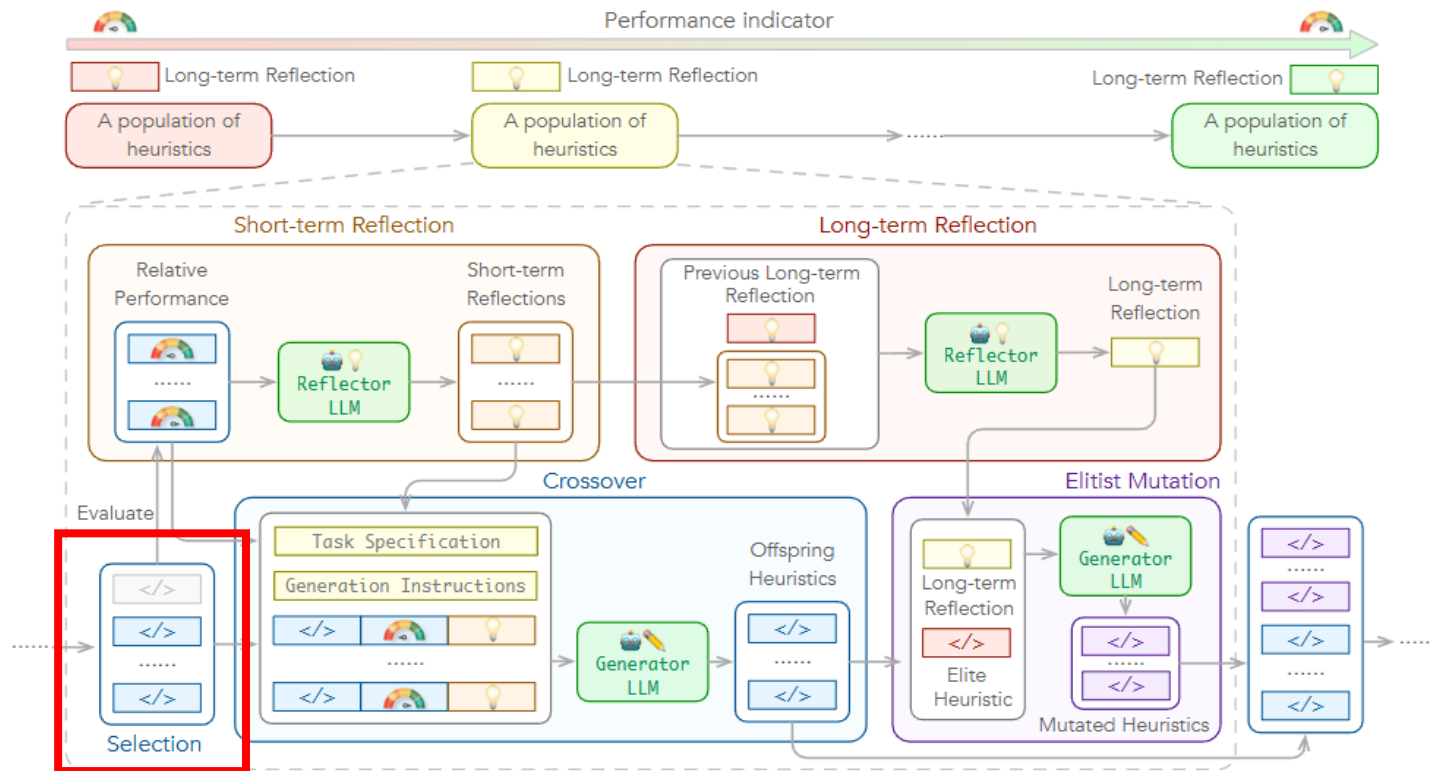
성찰 내용과 문제 명세를 바탕으로 실제 파이썬 코드를 작성하는 역할

❖ Reflector LLM:

코드 간의 성능 차이를 논리적으로 분석하여 개선 방향을 텍스트로 제공하는 역할

III ReEvo(Reflective Evolution)

❖ ReEvo(Reflective Evolution)



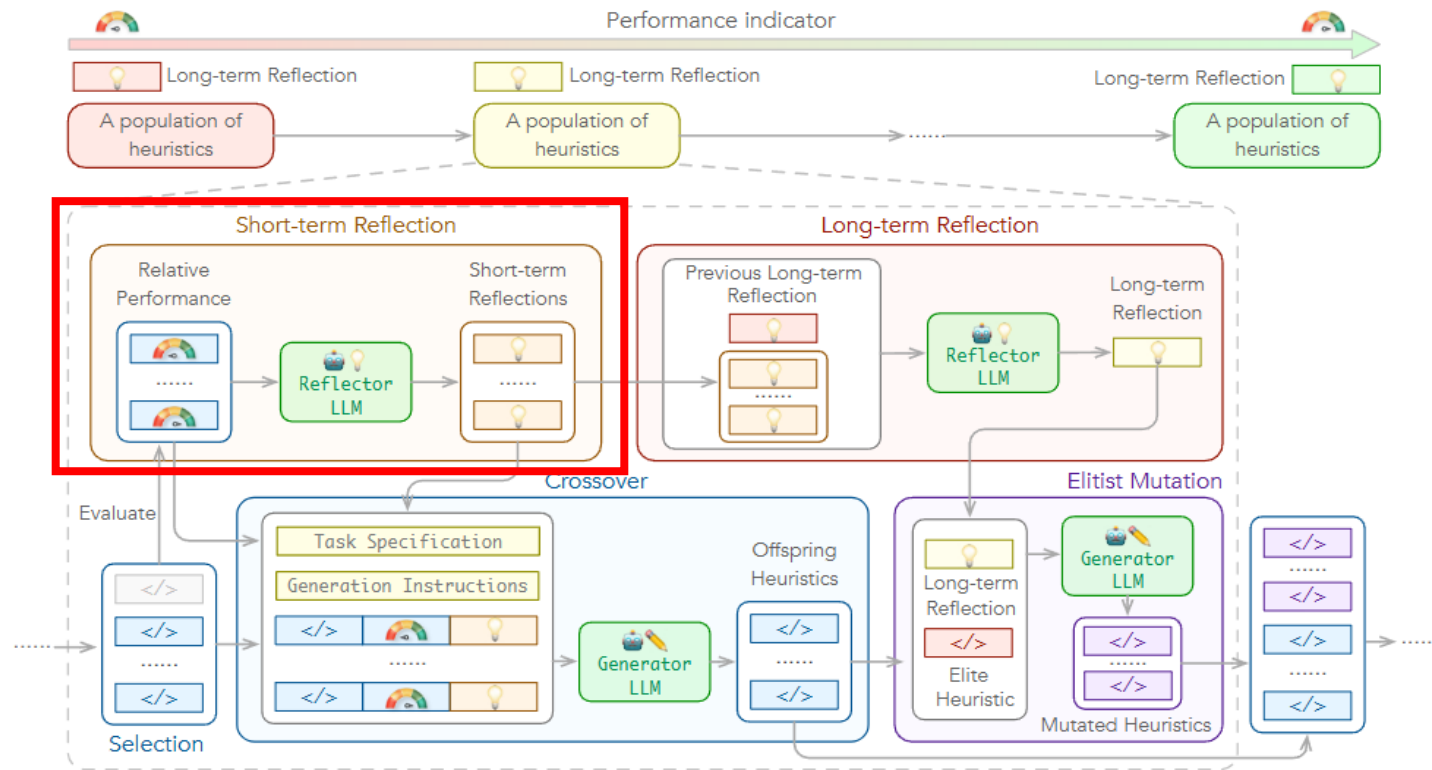
(a) ReEvo pipeline. **Top:** ReEvo evolves a population of heuristics. Insights and knowledge are verbalized as long-term reflections and accumulated throughout iterations. **Bottom:** A ReEvo iteration contains five sequential steps: selection, short-term reflection, crossover, long-term reflection, and elitist mutation.

❖ Selection:

- ✓ 실행에 성공한 휴리스틱 중 무작위로 두 개의 '부모' 코드를 선택, 이때 성능 수치가 동일한 쌍은 피함.

III ReEvo(Reflective Evolution)

❖ ReEvo(Reflective Evolution)



(a) ReEvo pipeline. **Top:** ReEvo evolves a population of heuristics. Insights and knowledge are verbalized as long-term reflections and accumulated throughout iterations. **Bottom:** A ReEvo iteration contains five sequential steps: selection, short-term reflection, crossover, long-term reflection, and elitist mutation.

❖ Short-term reflection:

- ✓ Reflector LLM이 두 부모 코드의 성능 차이를 분석하여 개선 힌트를 줌. 이전 연구들과 달리 ReEvo는 reflection을 진화 탐색에 통합하고 비교 분석하여 반영함. 이 방식을 통해 유전적 단서를 제공하고 탐색 공간 내에서 언어적 기울기를 제공.

❖ Relative Performance:

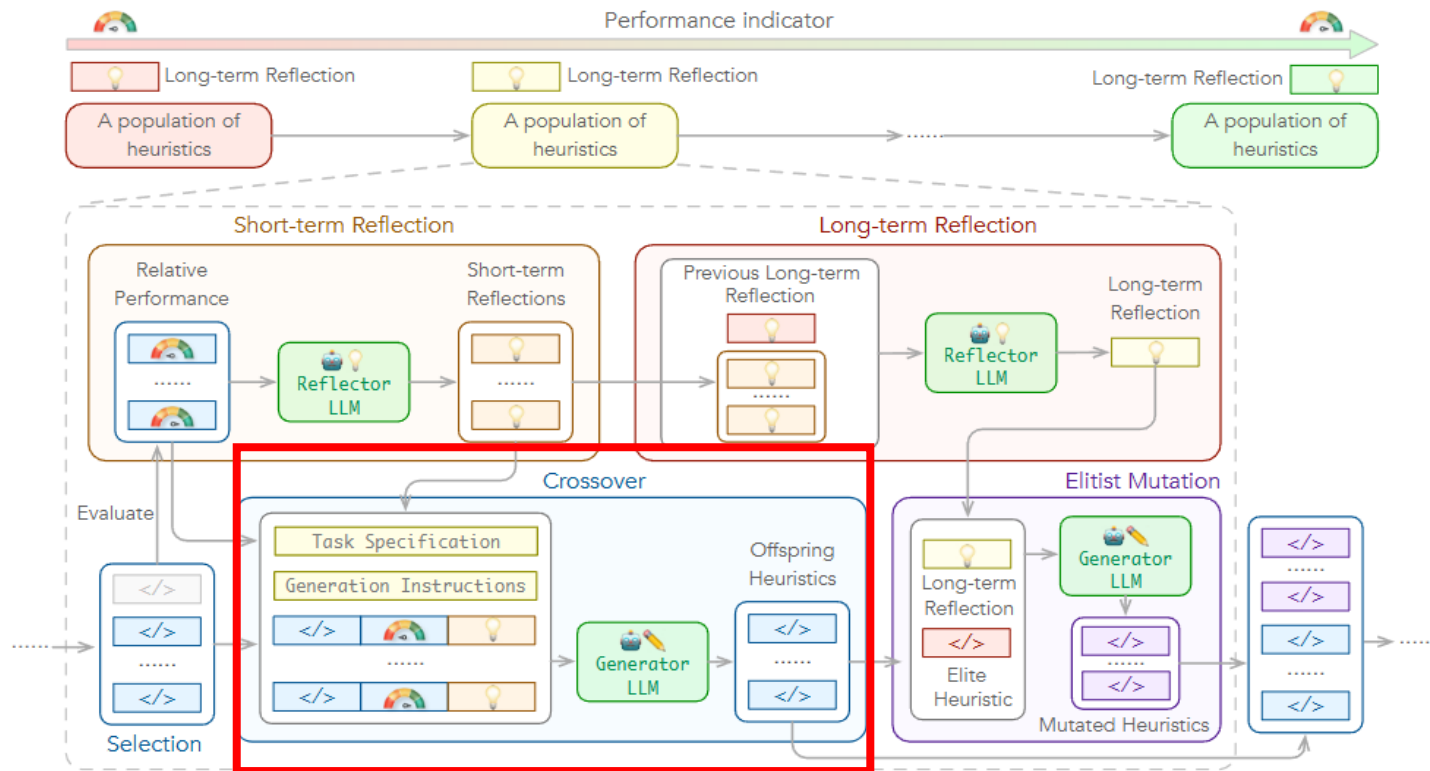
선택된 두 부모 휴리스틱 중 어느 쪽이 더 우수하고 열등한지에 대한 수치적/상대적 지표

❖ Task Specification:

해결해야 할 문제의 설명(예: TSP의 정의), 함수의 입력/출력 형식등을 담음

III ReEvo(Reflective Evolution)

❖ ReEvo(Reflective Evolution)



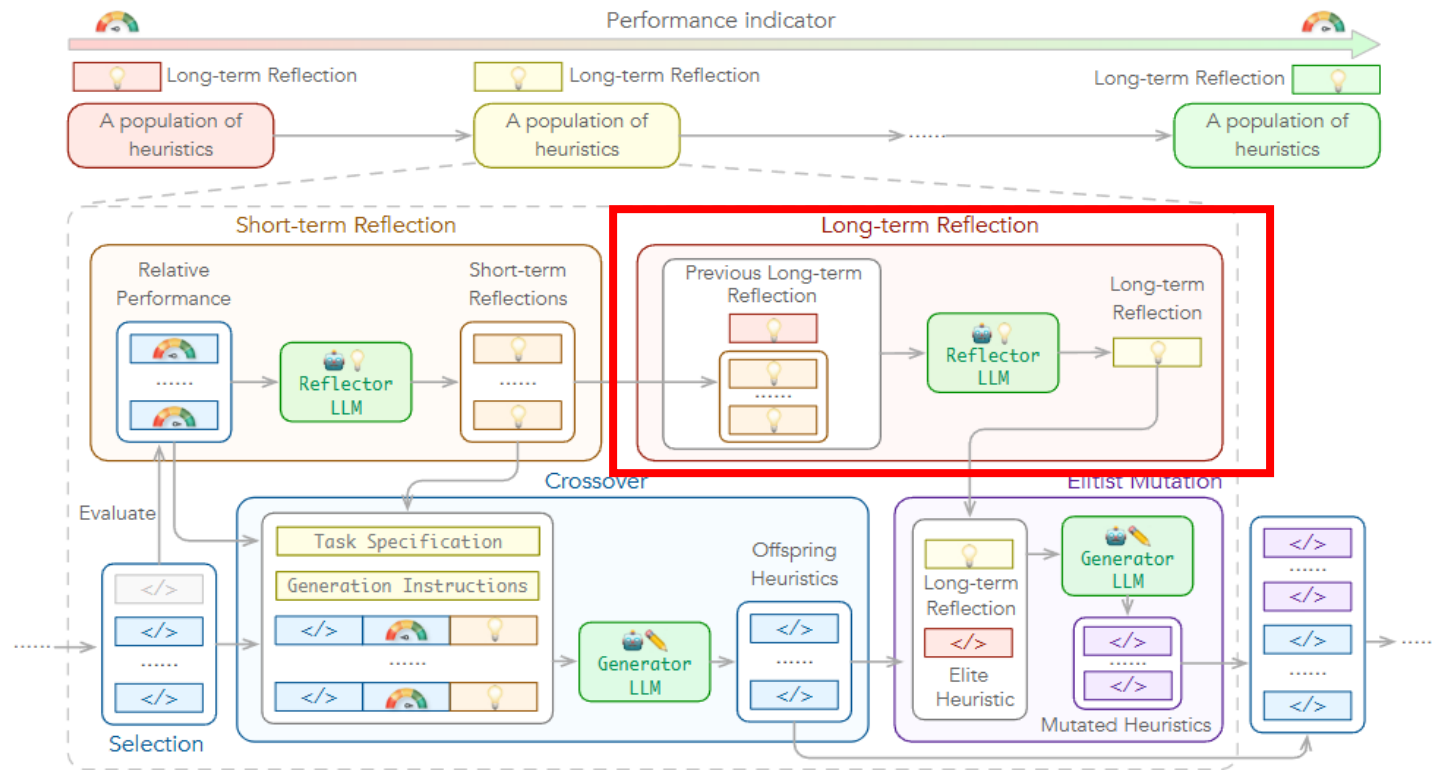
(a) ReEvo pipeline. **Top:** ReEvo evolves a population of heuristics. Insights and knowledge are verbalized as long-term reflections and accumulated throughout iterations. **Bottom:** A ReEvo iteration contains five sequential steps: selection, short-term reflection, crossover, long-term reflection, and elitist mutation.

❖ Crossover:

- ✓ Generator LLM이 부모 코드와 Short-term reflection내용을 참고해 새로운 '자손' 코드를 짤

III ReEvo(Reflective Evolution)

❖ ReEvo(Reflective Evolution)



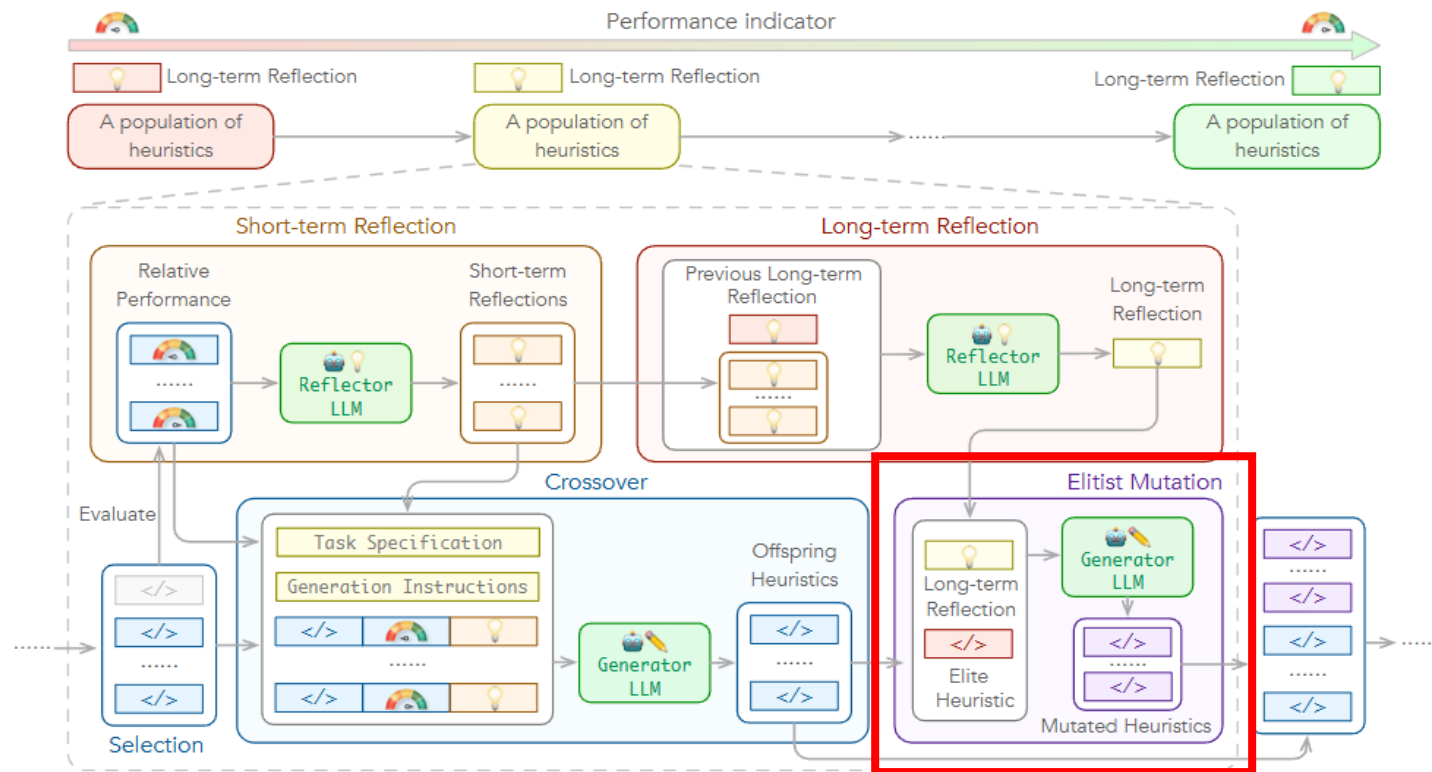
(a) ReEvo pipeline. **Top:** ReEvo evolves a population of heuristics. Insights and knowledge are verbalized as long-term reflections and accumulated throughout iterations. **Bottom:** A ReEvo iteration contains five sequential steps: selection, short-term reflection, crossover, long-term reflection, and elitist mutation.

❖ Long-term reflection :

- ✓ Reflector LLM이 이전의 장기적 성찰과 새로 얻은 단기적 성찰을 바탕으로 이를 요약하여 핵심 지식으로 축적. 메모리 과부하를 막는 필터 역할을 함

III ReEvo(Reflective Evolution)

❖ ReEvo(Reflective Evolution)



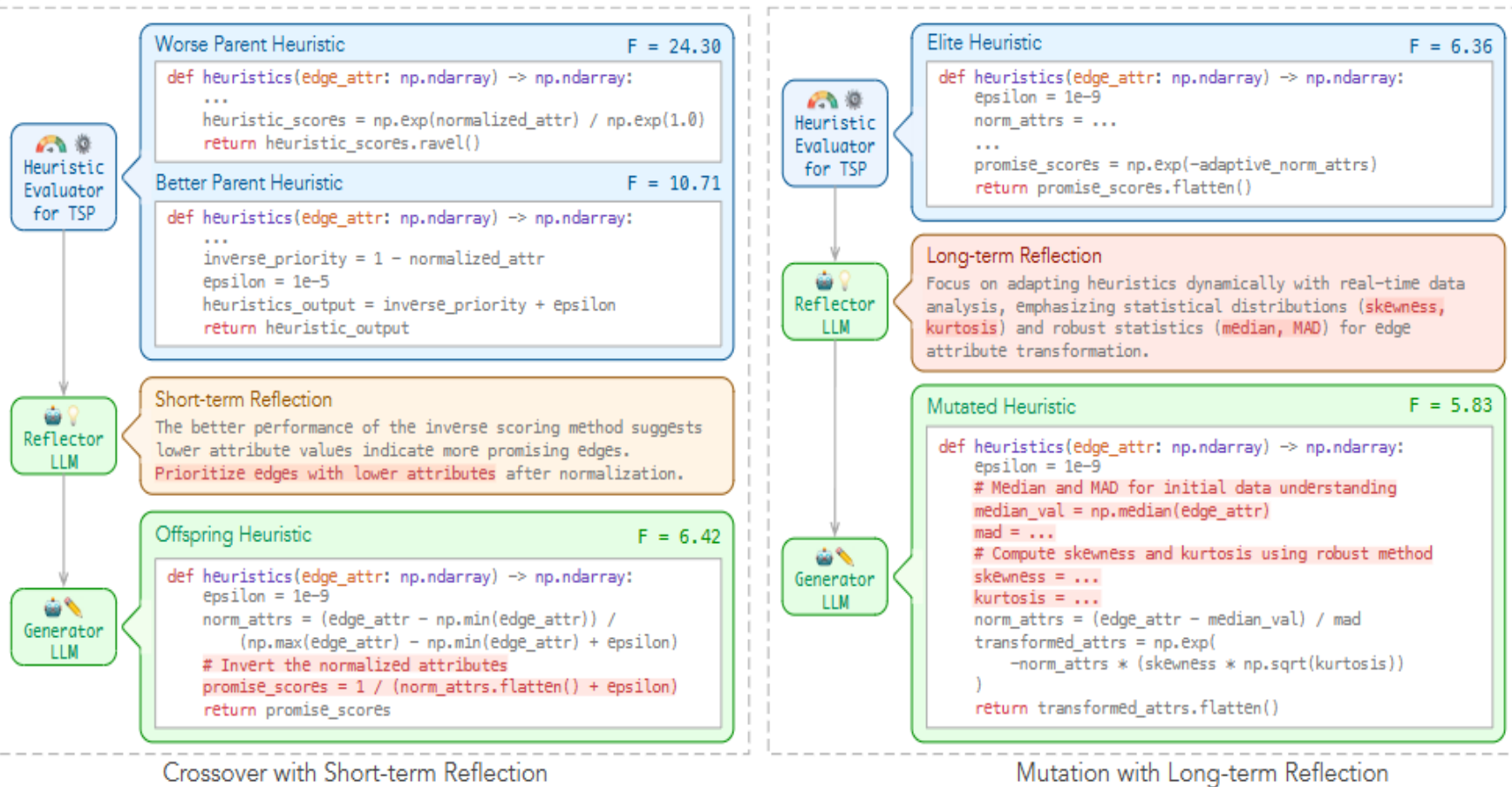
(a) ReEvo pipeline. **Top:** ReEvo evolves a population of heuristics. Insights and knowledge are verbalized as long-term reflections and accumulated throughout iterations. **Bottom:** A ReEvo iteration contains five sequential steps: selection, short-term reflection, crossover, long-term reflection, and elitist mutation.

❖ Elitist mutation:

- ✓ 현재까지 가장 우수한 코드를 장기 성찰 내용을 바탕으로 변형시킴. 현재 성능을 뛰어넘는 코드 생성을 위함

III ReEvo(Reflective Evolution)

❖ Examples of reflections for black-box TSP



❖ 예시:

✓ Short-term Reflection:

"역순 점수(inverse scoring) 방식의 더 나은 성능은 낮은 속성값이 더 유망한 엣지(edge)임을 시사합니다. 정규화 후 낮은 속성값을 가진 엣지에 우선순위를 두세요."

-> 언어적 기울기로 작용하여 속성값이 낮은 쪽을 선택하는 휴리스틱 로직을 생성

✓ Long-Term Reflection:

"실시간 데이터 분석을 통해 휴리스틱을 동적으로 조정해야 하며, 강건한 변환을 위해 중앙값(median)이나 MAD와 같은 통계적 분포에 집중해야 한다"

➔ 가이드를 따라 LLM이 변이 휴리스틱 생성

(b) Examples of reflections for black-box TSP. Heuristics are designed for Ant Colony Optimization (see § 5.2). **Left:** Given a pair parent heuristics, ReEvo correctly infers the TSP objective and generates a better offspring accordingly. **Right:** Given the elite heuristic and accumulated long-term reflections, ReEvo incorporates the suggested statistics and yields a better mutated heuristic.

Figure 1: An illustration of ReEvo.

IV Heuristic generation with ReEvo

❖ Penalty heuristics for Guided Local Search(GLS)

- ✓ GLS: 로컬 서치 중 국소 최적해에 빠졌을때 현재 해의 특정 기능에 패널티를 부여하여 새로운 탐색공간으로 유도
 - 예: TSP의 경우, TSP에서 선택된 특정 도시 사이의 경로에 패널티 부여, 현재 국소 최적해는 패널티에 의해 나쁜 해로 취급, 이후 새로운 영역 탐색 시작

❖ ReEvo의 역할

- ✓ 패널티를 결정하는 핵심 로직을 파이썬 코드로 생성
 - 예: 프롬프트 초기 작업 명세 작성시 ‘입력값은 엣지 속성이고, 출력은 패널티 점수여야한다’와 같이 제공, 결과물로 국소 최적해를 탈출시키는 최적 파이썬 함수 제공
 - 이렇게 생성된 휴리스틱을 본 논문에서는 KGLS-ReEvo라 지칭

Table 1: Evaluation results of different local search (LS) variants. We report optimality gaps and per-instance execution time.

Method	Type	TSP20		TSP50		TSP100		TSP200	
		Opt. gap (%)	Time (s)	Opt. gap (%)	Time (s)	Opt. gap (%)	Time (s)	Opt. gap (%)	Time (s)
NeuOpt* [52]	LS+RL	0.000	0.124	0.000	1.32	0.027	2.67	0.403	4.81
GNNGLS [24]	GLS+SL	0.000	0.116	0.052	3.83	0.705	6.78	3.522	9.92
NeuralGLS† [75]	GLS+SL	0.000	10.005	0.003	10.01	0.470	10.02	3.622	10.12
EoH [47]	GLS+LHH	0.000	0.563	0.000	1.90	0.025	5.87	0.338	17.52
KGLS‡ [1]	GLS	0.004	0.001	0.017	0.03	0.002	1.55	0.284	2.52
KGLS-ReEvo‡	GLS+LHH	0.000	0.001	0.000	0.03	0.000	1.55	0.216	2.52

*: All instances are solved in one batch. D2A=1; T=500, 4000, 5000, and 5000 for 4 problem sizes, respectively.

†: The results are drawn from the original literature. ‡: They are based on our own GLS implementation.

❖ Heuristic measures for Ant Colony Optimization(ACO)

✓ ACO

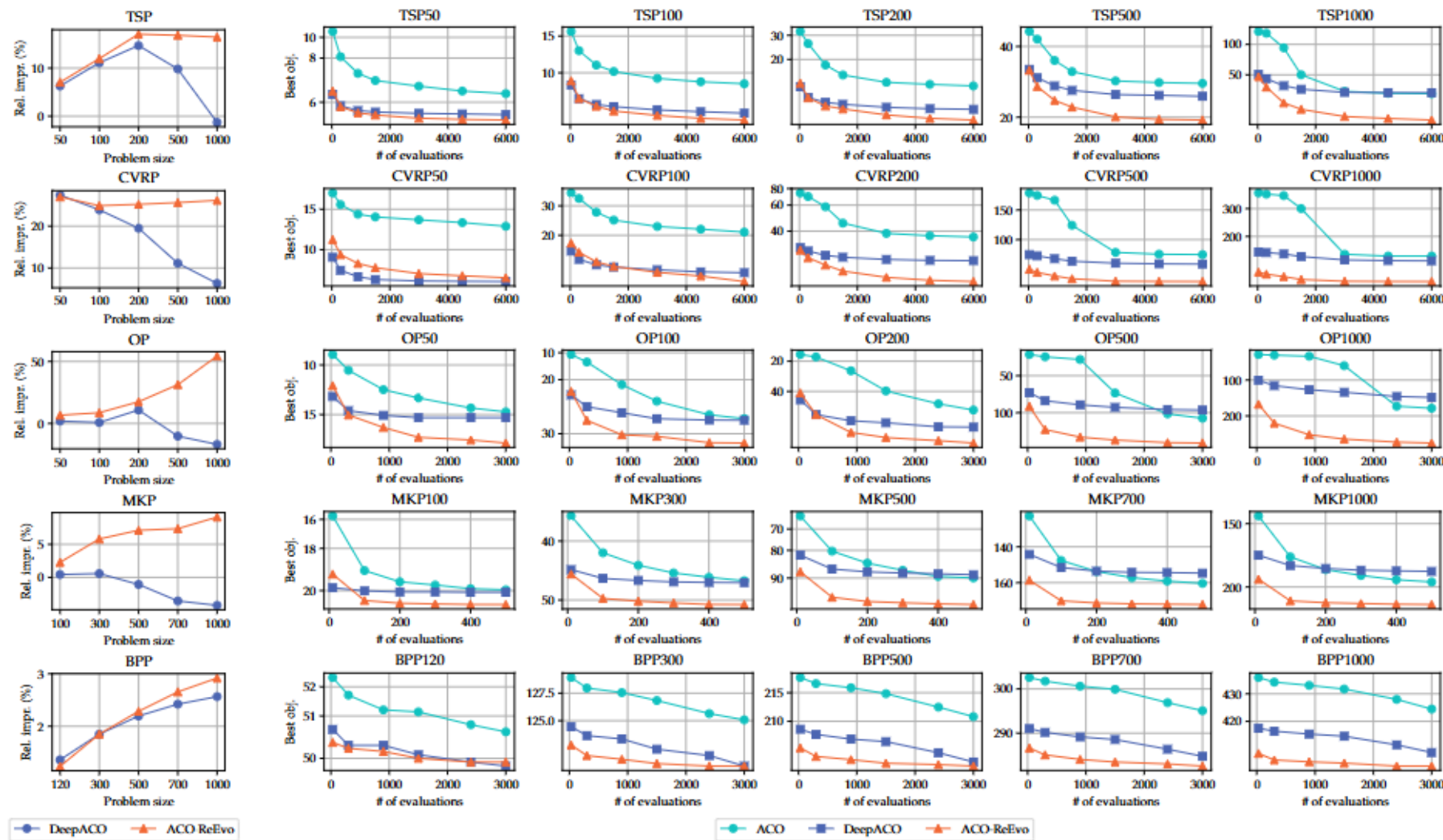
- 작동원리: 개미의 길찾기 방식에서 고안한 휴리스틱으로, 페로몬과 휴리스틱 척도를 결합한 확률을 사용하여 노드 사이를 이동
- 페로몬: 개미가 길을 지나며 바닥에 남기는 화학물질, 알고리즘에서 어떤 경로가 얼마나 성공적이었는지 기록하는 공유 메모리로, 긍정적 피드백으로 작용. 많은 개미가 지나간 길일수록 페로몬 농도가 진해짐
 - 증발: 시간이 지나면 페로몬은 조금씩 사라짐. 이는 지역최적해에 갇히지 않도록 도와줌
- 휴리스틱 척도: 페로몬과 상관없이, 개미가 현재 위치에서 목적지가 얼마나 가까워보이는지 판단하는 정보, 문제에 대한 사전 지식을 바탕으로 내리는 즉각적인 판단, 보통 거리의 역수를 사용하여 목적지와 가까운 곳의 점수를 높인다.

✓ ReEvo의 역할

- 휴리스틱 척도를 결정하는 핵심 로직을 파이썬 코드로 생성
- 예: CVRP의 경우, 노드간의 거리, 화물의 수요, 차량의 잔여 용량 등 변수를 조합하여 가장 지능적인 휴리스틱 척도를 계산하는 파이썬 코드 생성

IV Heuristic generation with ReEvo

❖ Heuristic measures for Ant Colony Optimization(ACO)



✓ 다섯 가지 다른 COP(TSP,CVRP,OP,MKP,BPP)에 대해 전문가 설계 휴리스틱 및 학습된 신경망 휴리스틱과 비교하여 일관된 우수성을 보임.

IV Heuristic generation with ReEvo

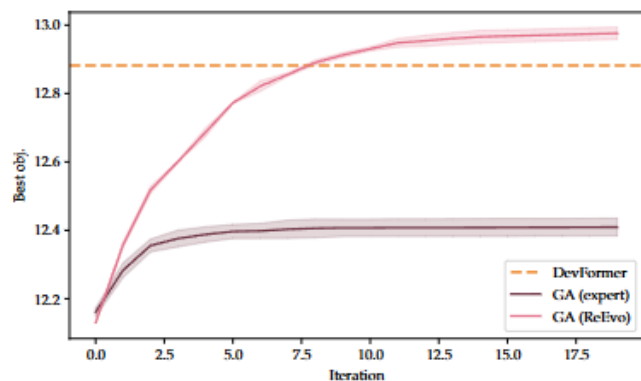
❖ Genetic operators for Electronic Design Automation

❖ GA

❖ 자연의 진화 원리를 모방하여, 수많은 후보중 최적의 해를 점진적으로 찾아나가는 탐색 알고리즘

❖ ReEvo의 역할

❖ 기존에 수동으로 개발했던 교차(Crossover) 및 돌연변이(Mutation) 연산자 로직을 파이썬 코드로 생성



Method	# of shots	Obj. $\uparrow$
Pointer-PG [30]	10,000	9.66 ± 0.206
AM-PG [60]	10,000	9.63 ± 0.587
CNN-DQN [61]	10,000	9.79 ± 0.267
CNN-DDQN [100]	10,000	9.63 ± 0.150
Pointer-CRL [30]	Zero Shot	9.59 ± 0.232
AM-CRL [62]	Zero Shot	9.56 ± 0.471
DevFormer-CSE [31]	Zero Shot	12.88 ± 0.003
GA-expert [63]	400	12.41 ± 0.026
GA-ReEvo (ours)	400	12.98 ± 0.018

Figure 3: **Left:** Comparison of DevFormer [31], the expert-designed GA [63] and our ReEvo-designed GA on DPP. The evolution curves plot the best objective value over generations; the horizontal line indicates the reward of end-to-end solutions generated by DevFormer. **Right:** Evaluation results of DPP solvers. We report the number of solution generations and the average objective value of 100 test problems.

❖ ReEvo가 설계한 유전 알고리즘이 전문가 설계 알고리즘과 최신 뉴럴 솔버를 능가하는 성능을 보여줌

IV Heuristic generation with ReEvo

❖ Constructive heuristics for the Traveling Salesman Problem

❖ Constructive heuristic

❖ 정해진 결정 규칙을 바탕으로, 복잡한 제약 조건을 만족시키며 해를 점진적으로 구축해 나가는 방법론

❖ ReEvo의 역할

❖ LLM이 현재 상태를 기준으로 지능적 결정 규칙을 파이썬 코드로 설계함

❖ 예 : TSP에서, '해당 도시가 남은 도시들로 부터 얼마나 고립되어 있는지 계산해서, 고립이 심하면 왕복이 힘들니 미리 방문하는 쪽에 가산점을 줘라 ' 와 같은 규칙을 설계함

Table 2: Comparisons of constructive heuristics designed by human, GHPP [15], and ReEvo. We report the average optimality gap of each instance, where the baseline results are drawn from [15] and the results of ReEvo are averaged over 3 runs with different starting nodes.

Instance	Nearest Neighbour	GHPP [15]	ReEvo
ts225	16.8	7.7	6.6
rat99	21.8	14.1	12.4
rl1889	23.7	21.1	17.5
u1817	22.2	21.2	16.6
d1655	23.9	18.7	17.5
bier127	23.3	15.6	10.8
lin318	25.8	14.3	16.6
eil51	32.0	10.2	6.5
d493	24.0	15.6	13.4
kroB100	26.3	14.1	12.2
kroC100	25.8	16.2	15.9

Instance	Nearest Neighbour	GHPP [15]	ReEvo
ch130	25.7	14.8	9.4
pr299	31.4	18.2	20.6
fl417	32.4	22.7	19.2
d657	29.7	16.3	16.0
kroA150	26.1	15.6	11.6
fl1577	25.0	17.6	12.1
u724	28.5	15.5	16.9
pr264	17.9	24.0	16.8
pr226	24.6	15.5	18.0
pr439	27.4	21.4	19.3
Avg. opt. gap	25.4	16.7	14.6

IV Heuristic generation with ReEvo

❖ Attention reshaping for Neural Combinatorial Optimization

❖ NCO에서 ReEvo의 역할:

- ✓ 딥러닝 모델이 학습결과 내놓은 어텐션 스코어를 새로운 제약이나 데이터 변화를 반영해서 추가적인 훈련없이 어텐션을 재조정하는 함수를 설계함
- ✓ 어텐션 스코어: 모델이 다음 행동을 결정하기 위해 현재 상태(Query)와 후보 대상들(Key) 사이의 유사도나 중요도를 계산한 점수

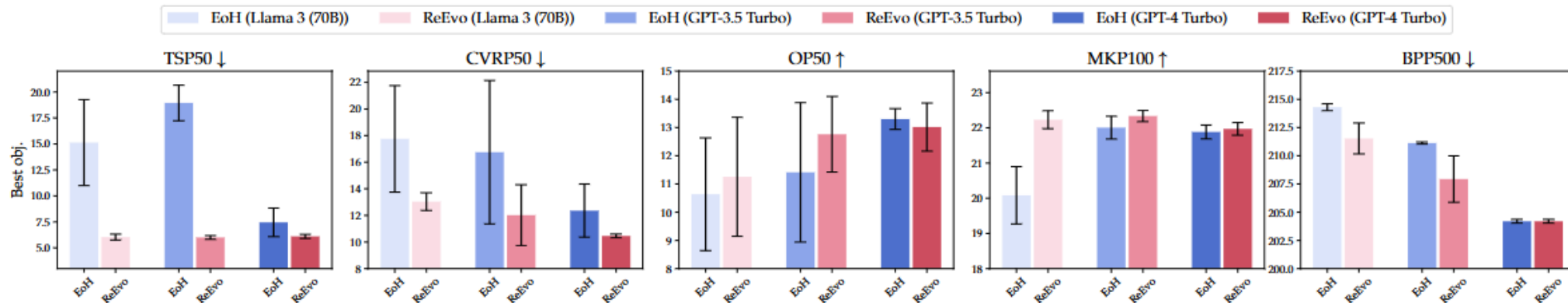
Table 3: Evaluation results for NCO solvers with and without different attention-reshaping heuristics.

	Method	$n = 200$		$n = 500$		$n = 1000$	
		Obj.	Opt. gap (%)	Obj.	Opt. gap (%)	Obj.	Opt. gap (%)
TSP	POMO [37]	11.16	4.40	22.21	34.43	35.19	52.11
	POMO + DAR [85]	11.12	3.98	21.63	30.95	33.32	44.05
	POMO + ReEvo [75]	11.12	4.02	20.54	24.32	29.86	29.08
	LEHD [51]	10.79	0.87	16.78	1.55	23.87	3.17
	LEHD + DAR [85]	10.79	0.89	16.79	1.62	23.87	3.19
	LEHD + ReEvo	10.77	0.74	16.78	1.55	23.82	2.97
CVRP	POMO [37]	22.39	10.93	50.12	33.76	145.40	289.48
	POMO + DAR [85]	22.36	10.78	50.23	34.05	144.24	286.37
	POMO + ReEvo	22.30	10.48	47.10	25.70	118.80	218.22
	LEHD [51]	20.92	3.68	38.61	3.03	39.12	4.79
	LEHD + DAR [85]	21.13	4.67	39.16	4.49	39.70	6.35
	LEHD + ReEvo	20.85	3.30	38.57	2.94	39.11	4.76

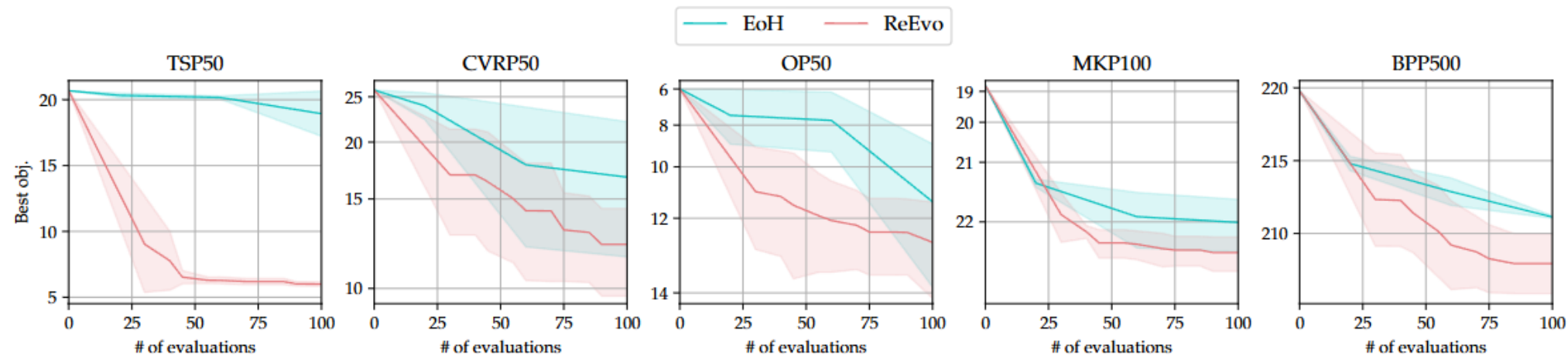
- ✓ ReEvo를 통해 LLM이 생성한 휴리스틱이 기존의 신경 조합 최적화 솔버의 성능을 크게 향상
- ✓ 복잡하고 대규모 문제에서 뛰어난 성능을 발휘

IV

Evaluating ReEvo



(a) LHH evolution results using different LLMs.



(b) LHH evolution curves using GPT-3.5 Turbo.

✓ 그림 (a),(b)는 다른 LHH인 EoH 와 비교하여 더 우수한 성능과 뛰어난 샘플 효율성을 제공하는것을 보여줌으로써 ReEvo의 우수성을 입증

❖ 한계점

- ✓ ReEvo의 성능은 기본적으로 사용되는 LLM의 코딩 능력과 논리적 추론 능력에 종속
- ✓ 평가(Evaluation)'과정에서 많은 시간이 소요
- ✓ 문제 설명이나 성찰(Reflection) 내용이 너무 길어질 경우, LLM의 Context Window제한에 걸릴 수 있음

❖ 결론

- ✓ 다섯가지 알고리즘 유형, 여섯가지 다른 COP에서 ReEvo를 적용하여 경쟁력있는 솔버 획득
- ✓ SOTA LHH와 비교했을때 우수한 샘플 효율성을 보여줌
- ✓ 더 넓은 응용 분야와 최적화 아키텍처에 적용 가능성